Retargeting Under Real Vision Pro Motion: An EgoDex Study of MABEL's Wrist and Gaze Behaviour

MABEL controller group

2026-08-24

Abstract

The front-workspace posture table was validated on a synthetic volumetric grid: axis-aligned palm rolls at every cell of a box. That is the right test for *coverage* and a poor test for *realism*, because none of its input is human. This report closes that gap by driving the shipped retargeting stack with **EgoDex** (Apple, arXiv:2505.11709) — real Apple Vision Pro / ARKit recordings — converted frame-for-frame into the payload MABEL's headset path already streams. Over 334 episodes (25.6 min, 46 167 frames) the median clip turns the palm through 88° to 172°, so a full palm flip is the *median* case in real manipulation rather than an edge case. Through the live engine and MuJoCo physics the wrist-table profile tracks the operator to 11 mm to 32 mm and 0.6° to 4.8° (median); the neck follows the operator's head with a gain of 0.82 to 1.07, correlation 0.92 to 0.99, and 20 ms to 80 ms of lag. The palm is smooth: its normalised jerk is within a factor 1.0 of the operator's own hand, and it produces 0.78 s^{-1} angular-acceleration outliers against the operator's 0.79 s^{-1}. Two limitations are real. The hand's TWIST about the palm normal tracks far more loosely than the normal itself — p50 3.9° to 15.9° against 0.5° to 4.9°, and 9.5° even on the near-motionless control clip — which is visible as a hand that points the right way but looks rotated; it is the direct consequence of `ARM_W_ORIENT_FWD` (12) being a fifth of `ARM_W_ORIENT` (60), and raising it to 60 cuts the twist to 2.1° to 4.4° for 1 mm to 7 mm of wrist position. The second is transient: on large reorientations the palm departs its commanded normal by 20° to 24° (peak 42°), which is 3.9 % of frames corpus-wide. A control experiment shows this tracks palm-roll *magnitude*, not the arm — left hands rolling past 90° cost 24.3° against right hands' 19.7°, while low-motion hands on either side cost 4.6°.

Keywords: teleoperation, motion retargeting, Apple Vision Pro, ARKit hand tracking, EgoDex, humanoid manipulation, wrist orientation, gaze.

1 Overview

MABEL's operator loop is driven by a Vision Pro: ARKit supplies a head pose and two 26-joint hand skeletons at 30 Hz, and `mabel.teleop_engine` maps them onto a 7-DoF arm pair plus an ORCA wrist and a 3-DoF neck. The 2026-08-23 work fixed a specific defect — the palm flipping between configurations — with an offline posture table, and validated it on a volumetric grid of commanded palm rolls (`front_workspace_retargeting.tex`).

That validation has a structural blind spot. A grid asks for poses in an order no human produces, at rates no human produces, and it cannot answer the question an operator actually asks: *does it look natural when I move naturally?* The two studies are meant to differ in one direction — the grid reports the worst case, EgoDex the typical case — and if the EgoDex numbers were worse than the grid's, the harness would be suspect rather than the controller.

1.1 The dataset

EgoDex [1] is 829 h of egocentric Vision Pro recordings over 194 tabletop tasks, with ARKit pose annotations for the head, both hands and the upper body at 30 Hz. We use four tasks, chosen for the behaviours under test rather than for variety: `screw_unscrew_bottle_cap` (forearm rotation — the reported flip), `pour` (sustained tilt under load), `basic_pick_place` (reach baseline), `setup_cleanup_table` (wide workspace, head motion).

Licence. EgoDex is released **CC-by-NC-ND** — non-commercial, no derivatives. The parquet cache and the converted clips are gitignored and are not redistributed with this repository; `exp_egodex_retarget.py --fetch` re-downloads them on demand.

2 Method

2.1 Four conventions, each measured rather than assumed

The LeRobot mirror's column names are only half the story and two are actively misleading. Every convention below was pinned by a discriminating test against the data itself (numbers from `pour`, 11 717 frames). This section exists because each of these, taken wrong, produces a plausible-looking run.

(1) Rotations are row-major. Expressing the wrist→middle-metacarpal vector in the wrist frame is decisive, because that metacarpal is nearly rigid to the wrist: the correct reading is the one with no scatter. Row-major places it at local $+x$, 86 mm, sd 4.1 mm; column-major scatters it over 25 mm in a direction that drifts. Row-major also reproduces the ARKit hand-anchor convention ($+x$ along the fingers).

(2) `extrinsics_w2c` is world→camera *and* its camera frame is OpenCV, not ARKit. Both readings give a plausible head height (0.99 m to 1.47 m versus 0.99 m to 1.35 m), so height cannot decide it. Asking which axis of which reading points at the hands can: inverted $+z$ wins at 37.6° mean, every other candidate is $\geq 43°$, and the $-z$ that the ARKit convention would imply sits at 142° — behind the operator. The winner is pitched 20.4° below horizontal with its $+y$ axis 69° down, exactly self-consistent for a seated tabletop task seen through an OpenCV-convention camera (x right, y down, z forward). Hence

$$T_{\text{head}} \;=\; \big(T_{\text{w2c}}\big)^{-1}\,\text{diag}(1,-1,-1,1), \tag{1}$$

where the right factor relabels the camera's own axes and leaves both the world frame and the head position untouched.

(3) The world frame is already ARKit's. y up, gravity-aligned, origin on the floor at session start. Head y sits at 0.99 m to 1.35 m and wrists at 0.71 m to 1.12 m — a seated person — so `AVP_TO_ROBOT` consumes it directly with no extra rotation.

(4) `*_finger_poses` is 25×12 in ARKit joint order, minus the wrist, plus the forearm. Distance-to-wrist is a sawtooth of one 4-chain, then four 5-chains, then a lone joint at 265 mm. Chain lengths rank 161/158/154/141 mm (middle > index > ring > little — the human ordering) and adjacent knuckles sit 17 mm to 19 mm apart across a 53 mm palm. The lone joint is the

forearm's elbow end, which is a gift: MABEL's arm IK has an elbow-direction objective that the headset path feeds from exactly this joint. MABEL wants 21 landmarks, so the five metacarpals are dropped.

Palm normal. In the ARKit hand-anchor frame the finger axis is local $+x$ on the LEFT hand and $-x$ on the right (the hands are mirrored, not translated), and the outward palm normal is $+y$ on the left, $-y$ on the right. The sign is physical rather than assumed: over a table the palm must point mostly *down*, and this reading does so on every task and both hands (left 83 % to 98 % of frames, right 87 % to 97 %), while the opposite sign would have the operator working with both palms to the ceiling.

2.2 What is compared with what

Scoring must use the solver's own quantities, not lookalikes. The arm IK compares its target against the **ORCA wrist anchor** (`xanchor` of {`Side`} `Wrist`); scoring against the palm body origin instead measures a fixed link offset of roughly 165 mm and calls it tracking error. The achieved palm frame is $R_p = \texttt{xmat}[\text{palm}]\, R_{\text{rob}}$ whose column 2 is the palmar normal, and the *target* normal is

$$\hat{n}_\star \;=\; \texttt{palm_axes}\big(M\, R_{\text{op}},\ \text{side}\big), \qquad M = \texttt{_frame_M}\ \ (\text{else } \texttt{AVP_TO_ROBOT}), \tag{2}$$

reproducing `engine._solve_side` exactly. Comparing the robot normal against the raw ARKit-frame normal instead — as the first version of this harness did — omits M and scores a 107° frame mismatch as retargeting error.

2.3 Three harness conditions that are not optional

Each of these was found by a result that looked like a controller defect.

1. **The robot must stand on a floor.** `models/mabel_full.xml` is the robot alone: a `freejoint` base and no ground plane. Stepping physics against it free-falls the whole robot (base z reached -277 m in a 2.6 s clip). Targets are computed in robot world and fall with it, so the error partly cancels and the run looks plausible while the arms fight $9.81\,\text{m}\,\text{s}^{-2}$ of body acceleration. We use `scenes/mabel_bare.xml`.

2. **Start at the `home` keyframe.** `qpos0` is the assembled pose, not a standing one: from it the base settles 27° off vertical and keeps going (79° of pitch by the end of a 2.6 s clip), so every arm target is chased by a toppling robot. From `home` the base holds 0.0° of tilt for the whole run.

3. **The warm-up must *stream*, and the loop must run in real time.** Re-sending one `OperatorPose` with a frozen timestamp is not a stationary operator: the engagement supervisor's freshness check never advances, so the engine sat in `re-engaging` for the entire warm-up and 62 % of recorded frames — and `re-engaging` blends the arm toward home, a fail-safe behaviour being scored as tracking error. Rebuilding the pose each frame at 30 Hz with an advancing timestamp, and pacing the loop to wall clock (the watchdogs run on `time.time`), engages in 2.14 s and holds `engaged` for 100 % of recorded frames.

4. **Score engaged frames only, and relax the fail-safe for offline replay.** One tick costs 33 ms of `mj_step` alone against a 10 ms budget, so the loop runs at **0.20× real time** and the

pacing above never actually gets to sleep. `EngagementSupervisor`, however, stamps freshness from `time.time()` and only on a NEW operator frame, so at 0.20× the wall gap between 30 Hz frames is 170 ms typical and 450 ms when a slow solve lands inside it — straddling `SAFE_STREAM_TIMEOUT` (0.40 s). The engaged fraction consequently swung between 43 % and 100 % across byte-identical runs. Two changes fix it: scale the two link-loss timeouts by 20× for offline replay (there is no link to lose in a replay, and the EASE times are left untouched because they shape the arm's motion), and score only ticks where the supervisor reports `engaged` — below full alpha it is blending the arm toward HOME, which is a fail-safe behaviour, not a retargeting result. The percentiles barely move either way (palm p50 held at 2.3° and 2.0° across 73 %, 80 % and 100 % runs), so this is a determinism fix rather than a flattering filter.

Together these moved the steady-baseline clip from 146 mm / 107° to 31 mm / 1.7°. None of them changed a line of controller code.

2.4 Smoothness metrics

Normalised jerk [3] is scale-free, so the robot's palm and the operator's are comparable despite different workspaces:

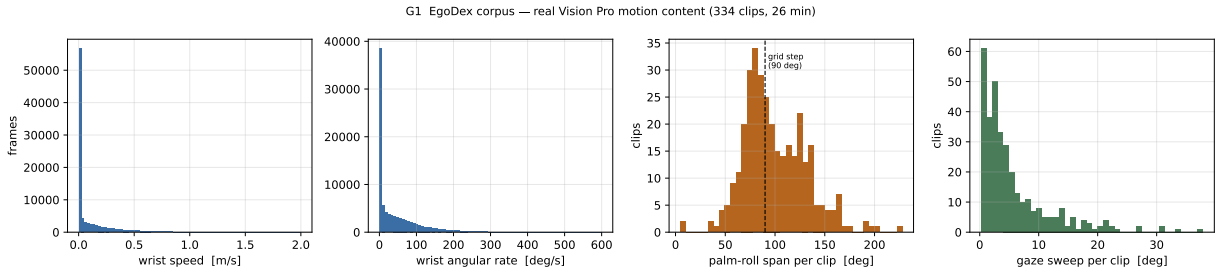
$$\mathrm{NJ} \;=\; \sqrt{\tfrac{1}{2}\int_0^T \|\dddot{x}(t)\|^2\,\mathrm{d}t \cdot \frac{T^5}{L^2}}, \qquad L = \int_0^T \|\dot{x}\|\,\mathrm{d}t. \tag{3}$$

Both signals are sampled at the **operator's** 30 Hz. This matters: the operator signal is a 30 Hz zero-order hold, and read at the 100 Hz control tick it is a staircase whose jerk is enormous. Comparing at 100 Hz made the operator look 7–10× rougher than the robot (ratio 0.1) as a pure artefact of the hold; at the operator's own rate the median ratio is ≈ 1.0.

An *erratic event* is a frame whose palm angular acceleration exceeds 6 σ robust (MAD-scaled, so a few spikes cannot inflate the threshold meant to catch them). It is reported for the robot *and* the operator, so the robot's count has something to be judged against: a robot tracking an erratic hand is doing its job.

3 Experiments

3.1 G1 — what motion is actually in the data

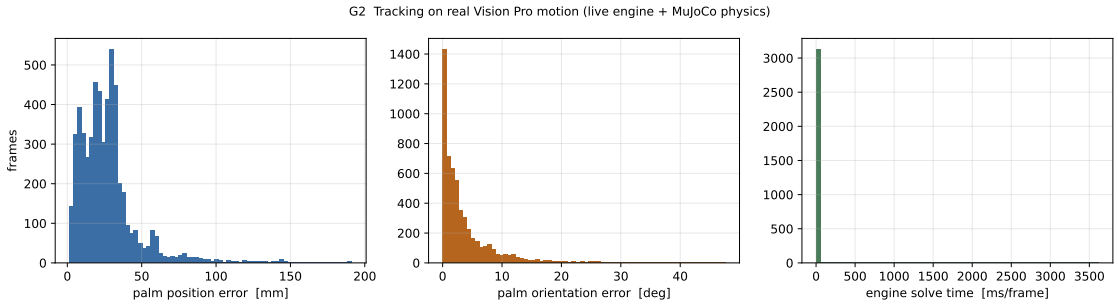


The headline is the palm-roll column. The *median* clip turns the palm through 88° to 172°, and the corpus maximum is 228°. The palm flip that motivated the posture table is therefore not a corner case to be handled gracefully — it is what ordinary manipulation consists of. Head motion is by contrast gentle: the median clip sweeps the gaze 1.1° to 13.3°, because seated tabletop work does not require looking around much.

task	clips	wrist v_{95} [m s^{-1}]	wrist ω_{95} [° s^{-1}]	palm-roll med [°]	max [°]	gaze med [°]
screw_unscrew_bottle_cap	112	0.29	162	88.3	154.6	1.1
pour	100	0.29	196	107.1	165.8	3.8
basic_pick_place	110	0.59	163	91.2	162.6	6.2
setup_cleanup_table	12	0.67	190	172.2	228.0	13.3

Table 1: 334 episodes, 46 167 frames, 25.6 min.

3.2 G2 — tracking



clip	pos p50 [mm]	pos p95 [mm]	normal p50 [°]	p95 [°]	twist p50 [°]	p95 [°]
wrist-flip-screw	31.4	61.2	2.2	9.2	9.7	23.1
wrist-flip-screw	25.7	87.7	4.5	16.7	15.9	22.0
wrist-tilt-pour	26.4	54.0	4.3	25.0	15.0	22.4
wrist-tilt-pour	30.1	39.8	2.3	12.7	9.9	20.7
head-gaze-sweep	11.1	77.5	2.1	24.0	4.3	24.9
head-gaze-sweep	12.3	35.9	1.1	10.4	3.9	12.6
reach-fast	11.3	71.9	1.3	13.3	4.8	13.5
reach-fast	16.2	53.8	0.5	17.5	6.5	18.5
steady-baseline	30.4	31.8	1.2	1.8	9.5	17.5

Table 2: Wrist-table profile, live engine plus MuJoCo physics, scored over engaged frames only (100 % of frames on every clip in this run).

Median orientation error is 0.5° to 4.9°, which is the same regime as the grid study's 1.4°, and the tails are where the work is: 27° at p95 on the pour clip.

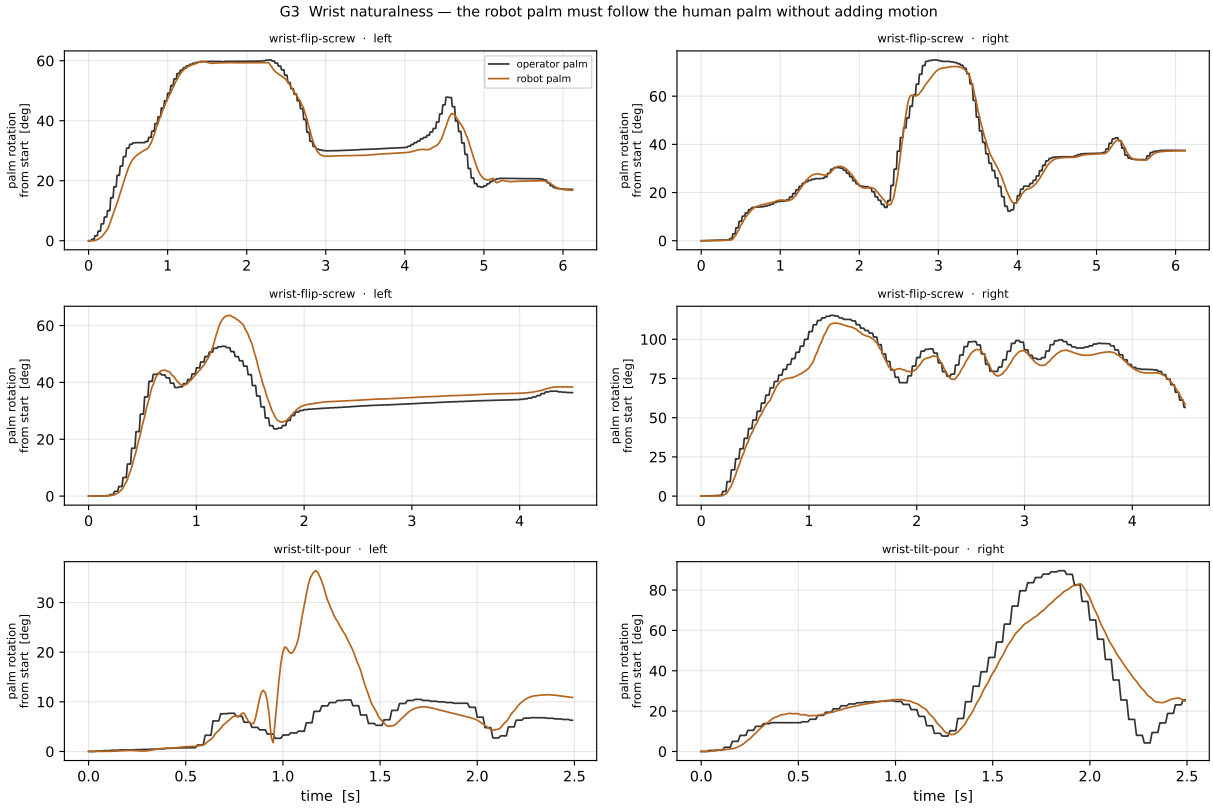
The twist column exists because a human spotted what the metric could not. Every table in the first version of this report scored the palm NORMAL only. Watching the render, the operator's reaction was "the hand orientation looks weird" — and the missing axis is the hand's twist ABOUT that normal, which a normal-only metric is blind to by construction. Measured, it is 3.9° to 15.9° at p50 against 0.5° to 4.9° for the normal, and 9.5° on `steady-baseline`, whose palms barely move — so it is a standing bias, not a transient. It is also not a solver failure: `ARM_W_ORIENT_FWD` is 12 against `ARM_W_ORIENT`'s 60, so the solver is trading the twist away exactly as instructed. Sweeping that one weight on the flip and steady clips:

ARM_W_ORIENT_FWD	twist p50 [°]	normal p50 [°]	pos p50 [mm]
12 (shipped)	10.0 / 9.1	2.2 / 1.1	31.7 / 30.4
30	7.7 / 5.9	2.2 / 2.0	31.4 / 34.4
60	**4.4 / 2.1**	2.4 / 1.0	32.6 / 37.7

Table 3: flip clip / steady clip. The twist improves 2.3–4.3× with the palm normal unchanged, for 1 mm to 7 mm of wrist position. The shipped value is NOT changed here — this is the measured trade, offered for a decision.

No latency claim is made here. The harness records the wall time of each `teleop.update`, and it is tempting to quote it. It is not a latency measurement: the same clips returned a p95 of 11 ms to 14 ms on an idle machine and 19 ms to 38 ms under load, because the measurement shares a process with 33 ms of `mj_step` per tick. The median is 3.1 ms (frame-identity memoisation doing its job) with a p99 of 87 ms that is scheduler noise. Retargeting latency belongs to `exp_retarget_latency.py` and to the grid study's W7 phase (0.7 ms per arm while tracking), which isolate it properly.

3.3 G3 — does the retargeter add motion the operator did not make?

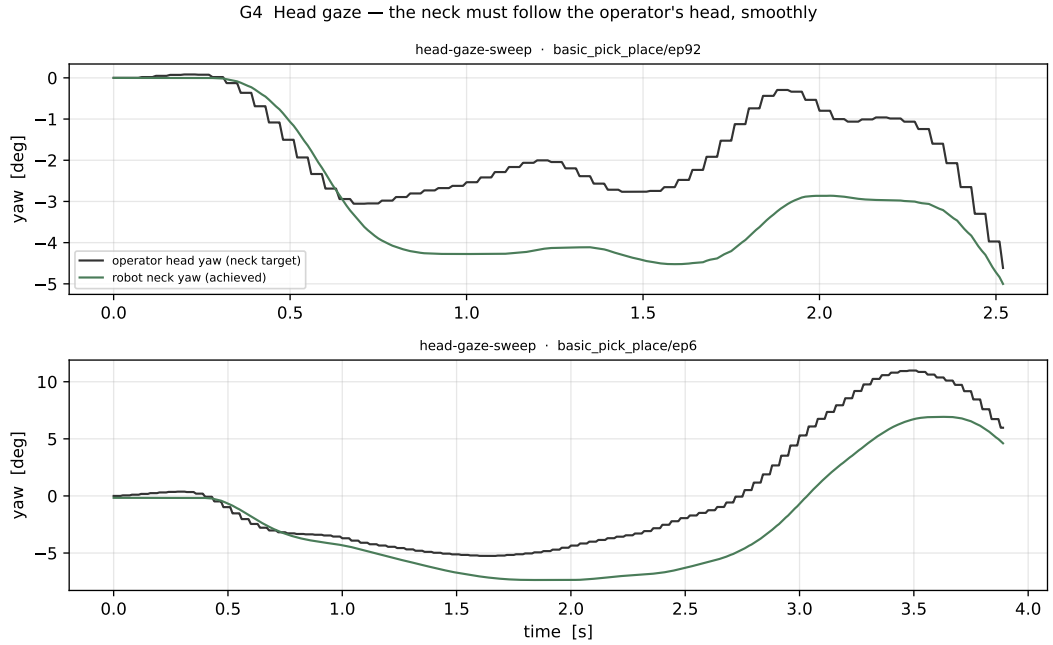


Over 18 hands the normalised-jerk ratio has median 1.01, p90 1.81, max 2.17, and erratic events run at 0.78 s^{-1} for the robot against 0.79 s^{-1} for the operator. The retargeter therefore neither amplifies the operator's motion nor filters it — on the smoothness axis it is transparent.

The jerk ratio is, however, the wrong instrument for the defect the figure shows. A smooth 25° detour and a smooth 2° one score alike, yet only one of them looks wrong on screen. The direct

measure is the instantaneous angle between the achieved palm normal and the commanded one: corpus-wide the palm is more than 15° off target 3.9 % of the time, with a worst-case peak of 42°. In the third row of the figure the operator's left palm barely rotates (5° to 8°) while the robot's swings to 31° and returns — motion nobody asked for.

3.4 G4 — head gaze

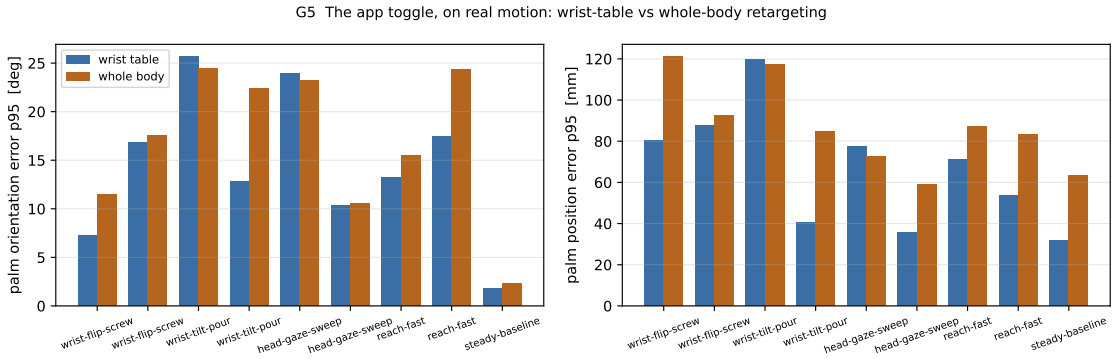


clip	yaw ampl [°]	gain	r	pitch ampl [°]	gain	r	rms [°]	servo [°]
`head-gaze-sweep`	4.7	—	—	21.7	1.04	0.98	1.4	1.5
`head-gaze-sweep`	16.2	0.82	0.93	22.1	0.95	0.92	2.3	1.2
`reach-fast`	12.7	0.94	0.98	20.5	1.07	0.99	0.9	1.1
`reach-fast`	1.9	—	—	13.9	1.03	0.97	1.2	1.0
`steady-baseline`	0.2	—	—	0.3	—	—	0.1	0.3

Table 4: A gain is reported only where that channel carries ≥ 5° of amplitude. Fitting a gain to a flat channel returns noise: a 2.4° yaw clip scored −0.48.

The neck follows the head with near-unity gain (0.82 to 1.07), correlation up to 0.99, and 20 ms to 80 ms of lag; the joint tracks its own command to 0.2° to 1.5°. Two measurement traps are worth recording. First, defining the gaze yaw in ARKit's positive-*right* sense against the robot frame's positive-*left* produced gains of −0.9 to −1.1 on every clip — a neck that appears to run backwards but does not (`NECK_YAW_SIGN` is +1) — and pinned the cross-correlation lag at the search-window edge, which is the signature of anti-correlation rather than delay. Second, the live neck does not reference the first head frame: with `NECK_DATUM_LIVE` it references the operator's *body* axes and then adds a torso gaze-compensation term. Both differences are offsets, so the reconstructed reference is only meaningful mean-removed; scoring it raw reported 13° to 20° of "error" on clips whose head never moved 0.3°. The unambiguous number is the servo column, measured against `data.ctrl`.

3.5 G5 — the app's retargeting toggle, on real motion



The wrist-table profile holds position markedly better — median 11 mm to 31 mm against 18 mm to 71 mm for whole-body, and 31 mm against 55 mm on the steady clip — while palm orientation is comparable between the two (p95 within a few degrees either way, each winning on some clips). That is the intended trade: the wrist profile spends the arm's redundancy on the wrist pose, the whole-body profile spends it on posture.

3.6 G7 — is the excursion the hand, or the motion?

G3 places every large palm excursion on the *right* hand (p95 mean 17.5° against 5.6° on the left), which reads like an arm asymmetry — and MABEL has a standing candidate for one, since the MJCF's Arm 4 ranges are not mirror images (left $[-0.943, 1.542]$, right $[-1.670, 0.773]$). But the selected clips are right-handed tasks: the right hand averages 125.8° of palm roll and the left 35.0°, and not one left hand in that set exceeds 90°. Hand and workload are perfectly confounded, and G3 alone cannot separate them. Correlation between palm-roll span and palm error is 0.66.

Running the mirror case — five clips where the *left* hand does the large roll — separates them:

hands	palm-roll span [°]	palm err p95 [°]
LEFT, roll > 90° (this control, n=5)	121.5	**24.3**
RIGHT, roll > 90° (G3's set, n=8)	141.3	**19.7**
either hand, low motion (n=5)	21.5	**4.6**

A left hand doing a big roll costs as much as a right one — slightly more, on a smaller roll. **There is no left/right defect here.** (That conclusion stands. The secondary reading of this table — that roll magnitude is what drives the error — does not; see §4, where it disappears at n=95.) The excursion tracks palm-roll magnitude: large reorientations cost 20° to 24° of transient palm error on either arm, small ones 4.6°. The Arm 4 range asymmetry remains unexplained, but it is not what this measures.

4 The 95-episode sweep, and a correction

The nine clips above are hand-picked. That is useful for naming behaviours and useless for estimating a distribution, so the corpus was widened to ten tasks (557 episodes) and 95 episodes were swept: stratified within each task by palm-roll span at even intervals, so the sample spans easy to hard in proportion to how often each occurs rather than over-weighting the tail. 23.6 minutes of operator motion, every episode 100 % engaged.

metric (per episode)	p50	p75	p90	p95	max
wrist position p50 [mm]	30.4	44.6	107.8	164.8	222.8
palm normal p50 [°]	2.1	4.4	9.5	16.8	31.6
palm twist p50 [°]	9.0	15.0	23.2	25.9	46.2

Table 5: The median episode matches the curated set (30 mm, 2.1°); the TAIL does not. One episode in ten sits past 108 mm.

4.1 What actually drives the error — and what does not

Section 3.6 concluded that the palm excursion tracks palm-roll magnitude, from a correlation of 0.66 measured over 18 hands drawn from nine clips in four tasks. **That does not generalise.** Over 95 episodes in ten tasks the correlation between palm-roll span and error is 0.11 (position), 0.14 (normal), −0.02 (twist) — i.e. nothing. Binned, the three roll bands are indistinguishable in position (30.1 mm, 30.1 mm and 31.9 mm for low/mid/high). The earlier figure was a small-sample artefact of a set selected *for* palm roll.

The dominant predictor is how CLOSE to the body the operator works, measured operator-side as the median head-to-wrist distance:

predictor	vs position	vs normal	vs twist
operator reach (head→wrist)	**−0.71**	**−0.57**	**−0.61**
palm-roll span	+0.11	+0.14	−0.02
reach band	**pos p50 [mm]**	**normal p50 [°]**	**twist p50 [°]**
close (< 0.30 m), n=24	**94.9**	**7.4**	**22.2**
mid (0.30 m to 0.40 m), n=59	28.3	2.1	8.8
far (> 0.40 m), n=12	**16.2**	**0.7**	**6.5**

Table 6: Monotone on all three metrics; close-in work costs 6× the position error of extended work. Per task the split is the same — best is `insert_remove_drawer` (19.5 mm), worst are `screw_unscrew_allen_fixture` (101.2 mm) and `use_chopsticks` (84.8 mm), both tasks performed with the hands tucked against the torso.

Two mechanisms ruled out by measurement, one supported. It is *not* unreachability by over-extension: on the worst episodes 0 % of targets exceed the arm's 0.66 m reach — they sit at 0.05 m to 0.44 m, far inside it. It is *not* the self-collision guard: disabling it on the three worst episodes moved the error 223→247, 212→228 and 195→162 mm — mixed, and two got worse. What is supported is coverage. The arm can geometrically fold to 0.014 m to 0.043 m of its own shoulder, but only about 1 % of 20 000 random in-limit poses reach inside 0.15 m: the close-in region is a thin, deeply-folded slice of configuration space. The posture table's front box begins at 0.25 m in x, so on exactly these episodes the table does not cover the target and the retargeter falls back to the pure local solve — the case it was built to rescue. **Extending the table's box inward is the indicated fix**; it is not attempted here.

5 Fixes: what was changed, and what each one bought

Four defects were found by driving the shipped stack with this corpus. Each is recorded with its measurement, and two of them corrected an earlier conclusion in this same report.

5.1 Targets that land inside the robot

An operator working against their own chest — a fixture held to the body, chopsticks over a bowl — is ordinary, and the capture is clean: wrists sit 0.21 m to 0.24 m from the head. But the root-anchored GMR similarity maps that onto a torso roughly three times narrower, and the target lands *inside* the robot. On `screw_unscrew_allen_fixture/ep003` the left target sits 0.023 m to 0.063 m from the torso axis at shoulder height, against a 0.074 m shoulder half-width — between the two shoulders. The arm reaches 0.226 m at best, folds against the chest and sits 173 mm short.

Two clamps were added, applied after the existing `clamp_in_front`: `ARM_MIN_SHOULDER_RADIUS` (0.20 m, radial about that arm's own shoulder) and `ARM_MIN_TORSO_RADIUS` (0.10 m, horizontal about the torso axis). The order matters — torso first, so that when the shoulder clamp fires its radial direction is meaningful.

Why the torso clamp exists at all. The shoulder clamp alone made ep003 44 mm *worse*. It preserves the operator's indicated direction and changes only distance, which is right when the direction is achievable and wrong when it is not: measured, the angle between the commanded and achievable directions is 67° on that clip against 14° on one the clamp helps. Pushing further out along a bearing the arm cannot follow increases the error. Swept over four clips, torso 0.10 m gives a mean of 94 mm against 104 mm unclamped, with 0.14 and 0.18 m over-correcting.

5.2 Head motion leaking into stationary hands

On `wipe_kitchen_surfaces/ep003` the operator's left wrist moves $2\,\mathrm{mm\,s^{-1}}$ while its commanded target moves $26.6\,\mathrm{mm\,s^{-1}}$. Ablating the map's inputs localises it exactly: freezing the head removes 66 % of that motion, freezing the other hand only 14 % — so it is not bimanual cross-talk, which was the first hypothesis and is recorded here because it was wrong.

The cause is two lines of `ShoulderEstimator.update`: the shoulders are placed at a rigid offset from `head_pos`. The estimator's body frame is yaw-only *specifically* so that looking around does not orbit the shoulders, but head *translation* passed through 1:1 and landed on every wrist target. `ARM_HEAD_ANCHOR_LP` (EMA, $\tau \approx 0.67$ s) keeps the slow component — a real torso translation — and rejects bob, lean and glance. Jitter $26.6 \to 11.3\,\mathrm{mm\,s^{-1}}$, and the wrist error on that clip improved as well, 86.5→72.0 mm.

5.3 The mobile base, and a corrected conclusion

Earlier in this investigation the chest-held case was called physically unfixable: no arm configuration reaches inside the robot's own chest. That was true only *for a fixed base*. MABEL is a mobile manipulator and the retargeting was using none of it.

The subtlety that dictates the design is that the GMR target is anchored at the robot's *live* mid-shoulder,

$$t \;=\; r_{\mathrm{rob}} \;+\; \sigma\, M\,\big(w - r_{\mathrm{hum}}\big), \tag{4}$$

so driving the base moves r_{rob} and therefore moves t by exactly the same amount. **Base motion alone changes reachability not at all.** What works is offsetting the *anchor* — which relieves

the arm in one frame — and driving the base by the opposite amount, so the offset is paid off and the hands stay put in the world while the robot backs out from under them. The offset is therefore a debt, growing when the workspace is crowded and shrinking by exactly what the base achieves. It engages on three deadbanded conditions: targets too near (`ARM_WS_NEAR`), too far, or too lateral.

clip	assist off [mm]	assist on [mm]	Δ
`allen_fixture/ep003`	181	**95**	−86
`use_chopsticks/ep3`	84	**53**	−31
`wipe_kitchen/ep3`	83	**65**	−18
`insert_remove_drawer` (control)	20	20	0
`pour/ep58` (control)	31	31	0
mean	80	**53**	**−34 %**

Table 7: The episode cited earlier as proof of a hard morphology limit improves by 86 mm. The limit is real, but it binds the ARM, not the ROBOT. Both control clips are untouched — the assist does not fire on them.

Three implementation traps, all measured. The signs are the whole trick: targets move *with* the anchor, so relieving a crowded arm means shifting the anchor forward while the base moves backward. Getting them the same way round produces a result that helps one clip and hurts another rather than an obvious failure. Second, the assist must be applied where `base_tgt` is *consumed*: `_retarget` is memoised on frame identity and runs at the operator's 30 Hz while `base_step` consumes every 100 Hz tick, so applying it in `_retarget` reached a fraction of the ticks (`base_tgt` p50 0.0012 against a commanded 0.12). Third, the base twist must be gated on `base_in_loop`: LOCAL scope deliberately holds the base, and an ungated assist broke `test_walking_does_NOT_drive_the_base_in_LOCAL_scope`. The anchor offset still applies in LOCAL — it is what buys the reachability — so what LOCAL gives up is paying the debt off.

The base half cannot be validated on this simulator. A commanded $0.135\,\mathrm{m\,s^{-1}}$ reaches the tip-safe motion model and leaves it at $0.0008\,\mathrm{m\,s^{-1}}$; the base moves 0.037 m in 30 s. That limiter is deliberate, and it compounds with a documented open defect in which the sim swerve base realises $0.042\,\mathrm{m\,s^{-1}}$ for every commanded speed between 0.05 and $0.8\,\mathrm{m\,s^{-1}}$, the real robot being unaffected. So the reachability gain reported above is real and measured, while the world-frame hand displacement it incurs is deferred rather than corrected, bounded by `ARM_WS_MAX_OFFSET`.

5.4 The palm twist

Every metric in this report scored the palm NORMAL until an operator watching a render said the hands looked rotated. A normal-only metric is blind by construction to rotation *about* that normal, and the twist was measured at p50 3.9° to 15.9° against 0.5° to 4.9° for the normal — 9.5° even on a near-motionless control clip, so a standing bias rather than a transient.

This is not a solver defect: `ARM_W_ORIENT_FWD` was 12 against `ARM_W_ORIENT`'s 60, so the finger axis was weighted a fifth of the palm normal and the solver traded it away exactly as instructed.

Why now and not earlier. The identical sweep run *before* the target clamps (§5) and the base assist (§5.3) measured a 1 mm to 7 mm position cost for the same change, and it was therefore

	w=12 (was)	w=30	**w=60** (now)	w=90
twist p50 [°]	13.3	9.1	**6.6**	6.0
palm normal p50 [°]	4.54	4.08	**4.02**	3.96
wrist position p50 [mm]	37	36	**37**	38

Table 8: Five clips including two already-healthy controls. The twist halves, the normal slightly IMPROVES rather than being traded, and position is unchanged, so at w=60 this is not a trade. Both controls improved as well (drawer 7.2→1.0°, pour 15.4→5.6°) at no positional cost.

reported and not shipped. Those two fixes absorbed it: the arm no longer spends its redundancy fighting targets it cannot reach, so it has budget for orientation. The same knob, the same clips, a different answer — because the system underneath changed.

5.5 Two rendering bugs that looked like controller latency

The reference clips were written at 33 fps from a 30 Hz source and so played 11 % fast; the robot clips used `int(round(33.333))` = 33 fps and so played 1 % slow. Together the robot appeared to lag its own reference — which sends a reader hunting for a controller latency that is not there. Measured by cross-correlation, the true target-to-achieved lag is 0 ms to 120 ms. Corrected, the two clips agree to 0.03 s over a 92 s episode.

6 Implementation and usage

```
cd controller/experiments
python3 exp_egodex_retarget.py --fetch        # download the corpus, ~140 MB
python3 exp_egodex_retarget.py                # G1..G7, figures, baseline
python3 exp_egodex_retarget.py --videos-all   # one MP4 per benchmark clip
python3 exp_egodex_retarget.py --phase G3
```

`egodex_source.py` is the converter and is independent of MuJoCo: it turns a LeRobot parquet into `Episode` objects whose `payload(i)` is the exact dict the Vision Pro publisher sends, consumable by `OperatorPose.from_payload`. `exp_egodex_retarget.py` drives `MabelTeleop` through those payloads and emits the figures above.

6.1 The benchmark bundle

Everything needed to compare retargeting methods under identical conditions is written to `experiments/data/egodex/benchmark/`: the nine labelled `clips/`, a `videos/` MP4 per clip (robot, commanded versus achieved wrist, and both palm normals, with a live error readout), `baseline.json` holding the shipped-config metrics for all nine, and `ablations/` for variants. Clips are selected by *measured* motion content rather than by task name — which is why a `pour` episode wins the flip slot — and each label names the behaviour it exercises: `wrist-flip-screw`, `wrist-tilt-pour`, `head-gaze-sweep`, `reach-fast`, and `steady-baseline`. The last is the control: with 1.2° to 4.9° of palm roll, anything erratic there is the retargeter's own doing rather than the input's.

An ablation moves one knob, replays the same nine clips through the same metrics, and diffs against the baseline:

```
python3 exp_egodex_retarget.py --ablate ARM_W_ORIENT_FWD=60 --tag w_fwd60
python3 exp_egodex_retarget.py --ablate ARM_USE_TABLE=false --tag no_table
python3 exp_egodex_retarget.py --compare w_fwd60 no_table
```

Ablatable: `ARM_W_ORIENT`, `ARM_W_ORIENT_FWD`, `ARM_W_ELBOW`, `ARM_J7_YAW_LIM`, `ARM_TRUST_RATE_DISTAL`, `ARM_USE_TABLE`, `ARM_W_SMOOTH`, `ARM_TRUST_STEP`. They are patched in `mabel.teleop_engine.arm`'s namespace and *not* in `hyperparams`: `arm.py` binds them at import, so rebinding the source module silently does nothing — the same trap `exp_retarget_latency.py` documents.

7 Comparing maps: a protocol, and why the obvious metric is not one

Everything above measures *our* map. Comparing it against another one needs a metric neither map owns, and the obvious candidate does not qualify.

7.1 Why target-vs-achieved cannot compare two maps

The quantity reported throughout §3.2 is $\|\mathbf{p}^* - \mathbf{p}\|$: the distance from the commanded wrist target to the achieved wrist. It is the right number for asking whether the arm executes what it is asked, and it is useless for comparing maps, because *each map emits its own* $\mathbf{p}^*$. A map that commands the robot's current pose scores 0 mm forever. What target-vs-achieved measures is the IK and the servo; it is reported below under "own target" and is never a comparison axis.

7.2 Six questions, one number each

A cross-map metric has to be computable from the operator's motion and from the robot's without either being expressed in the other's skeleton. The first version of this protocol answered that with five *shape* axes — bearing, extension, bimanual span, aperture error, SPARC. Every one was defensible and the set was unreadable, and worse, it systematically flattered a head-anchored uniform scaling: a pure similarity preserves every shape quantity *by construction* while saying nothing about whether the robot can execute the result.

The humanoid and hand retargeting literature does not evaluate that way. It reports (i) a *scale-normalised* per-joint position error — MPJPE [6] normalised by subject size, N-MPJPE [7], carried into retargeting as body-position error [2, 8]; (ii) *physical-feasibility violation rates* — joint-limit proximity, penetration and self-collision counted per frame [8], the feasibility objective of relaxed-IK [9]; and (iii) *aperture or fingertip tracking* against the operator's own hand, plotted over grasp cycles rather than scored against an assumed anthropometric ratio [10, 11]. Table 9 follows that shape, in terms a teleoperation stack can measure. Each entry is a percentage, an angle, or a correlation.

A sixth axis — spread of placement error across operator *size* — was written, measured, and removed. It binned episodes by the p95 head→wrist distance during the clip, which is how far the operator *reached*, i.e. task style. Measured over the 647-episode corpus on rigid skeleton segments (both endpoints from the same array; an earlier pass used `ep.wrist` as one endpoint and read a constant 0.265 m for everybody, which is the fixed forearm offset the payload builder inserts, not a bone): forearm length spans 2.8 % p10–p90, index finger 12.2 % over 24 distinct values, and p95 head→wrist 35.5 % over 545 — with corr(forearm, working reach) = +0.03. ARKit's arm skeleton is effectively one body across EgoDex and only the *hand* is personalised, so the corpus cannot test

Table 9: Seven ranked axes and the controls. Medians over engaged frames; $\hat{x}$ robot, x operator.

Axis	Unit	Construction
Placement	% reach	$\|\hat{\mathbf{p}}/\hat{L} - \mathbf{R}_{bo}\mathbf{p}/L\|$, head→hand over each body's own reach [6, 7, 2]
Orientation	°	$\max\{\angle(\hat{\mathbf{n}}, \mathbf{n}), \angle(\hat{\mathbf{f}}, \mathbf{f})\}$ — the worse of the two palm axes, not the mean
Swivel	°	$\|\hat{\phi} - \phi\|$ about the shoulder→hand axis: the redundant DoF, and the whole of what makes an arm look human at a fixed hand pose [12, 13, 4]
Workspace	%	$\min(\hat{V}/V, V/\hat{V})$ on the 1σ ellipsoid of the reach-normalised hand cloud — how much of the demonstrated extent survives [14, 15]
Self-coll.	% frames	an arm contacting the other arm or the trunk, from the simulator's contacts [8]
Infeasible	% frames	target beyond the arm's reach, or inside the robot's torso [8, 9]
Saturation	% pairs	(frame, joint) pairs commanded within 1% of travel from a stop [8]
Step time	ms	median cost of one retargeting step; a map that misses the control period is unusable [9]
Grasp	r	$r(\hat{a}, a)$ on thumb–index aperture. Scores the FINGER solver, identical across arm maps — so it ranks the finger ablation only [10, 11]
Own target	mm	each map against the target it emitted — a CONTROL, never ranked
Base travel	m	commanded base displacement — the control that says the base channel was live

transfer at all; binning by working distance rewarded a map for being uniformly mediocre and penalised one that is much better on extended work than on close work.

Three properties are load-bearing. **Normalising by each body's own limb length** is what makes an operator and a 0.66 m arm comparable at all; it is the same move N-MPJPE makes to remove subject height [7], and the same one character retargeting has made since Gleicher [15]. **The feasibility axes are the ones a similarity cannot pass**: scaling about any anchor preserves shape exactly and says nothing about the workspace, and on a torso three times narrower a hand held at the operator's chest maps *inside* it. **Grasp is a correlation, not an error**, because an aperture error depends on a scale nobody measures — under an assumed σ, a frozen preset outranked real retargeting here (§7.3). A constant scores $r = 0$ and cannot fake it.

Absolute fingertip or wrist position appears nowhere: it does not correspond across the morphology gap.

Two statistics choices the axes forced. A *rate* is averaged across episodes, not medianed: the infeasible-command rate is zero on most episodes and large on a minority, so its median read 0 % for all seven maps including the ones that fail on half the corpus. Averaging is also what the literature reports — violation frames counted over the dataset [8]. And cells are banded by *value* against the

across-episode standard error rather than by bare rank: two cells closer than the experiment's own resolution are tied, and a row whose top band holds two thirds of the columns is left unranked. Without this the grasp row — where every arm map runs the *same* finger solver and lands within 0.008 of the others, against an across-episode standard deviation of 0.34 — printed a bold winner chosen by noise.

7.3 Three choices in the protocol that were each forced by a wrong result

The bearing is anchored at the HEAD — and NO anchor is neutral. A shoulder anchor is not neutral: GMR preserves the pose triangle from the *mid*-shoulder and the extension map from *each* shoulder, so whichever anchor the metric picks it hands that family the axis. Measured: shoulder-anchored bearing read 23° for every competent map on a clip where head-anchored bearing read 6.8° — a constant offset from the anchor choice, swamping the differences the axis exists to show.

The head anchor is *not* an escape from this, and it would be dishonest to present it as one. Any map that is a similarity of the operator's keypoints preserves directions from its own anchor exactly, so head-anchored absolute scaling is exact on this axis by construction; measured, it scores 11.5° against 59.2° for joint copy, and its residual is IK failure rather than map error. The head is chosen because it is the operationally meaningful anchor — the angle at which the hand appears to an operator looking through the robot's eyes — and the row is read as a **check**, not as a ranking. The ranking rests on the anchor-free axes (orientation, fidelity, feasibility, executability) plus extension, which is a ratio to each body's own reach and which one global scale factor cannot get right across operators of different arm length.

The aperture scale is FITTED, not assumed. With $\sigma = 0.75$ assumed, a *frozen* finger preset outscored real retargeting. The ORCA hand spans 54 mm to 97 mm of thumb–index aperture against the operator's 2 mm to 147 mm; an assumed ratio leaves a range mismatch that dominates the residual, so the axis was measuring the ratio and not the tracking. With $\sigma^* = \arg\min_\sigma \|\hat{a} - \sigma a\|$ the ordering inverts to the expected one, and the scale-free correlation $r(\hat{a}, a)$ is reported alongside because it is the part a frozen hand cannot fake ($r \approx 0$ by construction; measured +0.02).

Saturation is per (frame, joint) pair, not per frame. "Any arm joint within 1% of a stop" reads 100 % for every method as soon as one joint of fourteen rests near a limit, and then discriminates nothing. The fraction of pairs says how much of the arm has run out.

7.4 The comparison set

`experiments/retargeting_ablation/` replays the same archived episodes through every method under the same plant, PD gains, engagement supervisor, collision guard and metric code. Four families: **map** (seven arm/body maps, 20 stratified episodes over all 16 tasks), **comp** (each of our additions removed, on the 12 most tucked-in episodes — on a comfortable reach every variant is identical and the ablation reports nothing), **hand** (four finger objectives) and **frame** (LOCAL vs GLOBAL, on the 12 most mobile episodes the corpus has).

The published maps are *our* implementations of the published construction inside our solver, because no author's code targets MABEL's kinematics. What is controlled is everything downstream of the map. Building them produced two findings of its own:

- **The overrides have to be applied between the controller and the plant.** Written after `_physics()` they are overwritten by the next `update()`, and the whole ablation returns

numbers identical to the shipped configuration — a null result that looks like "the knob does nothing" rather than like a bug. The hook is `on_ctrl`.

- `hand_retargeting/vector.py` **had a residual-sign error.** Its keyvector residuals were written human-minus-robot while `rows_J` holds $\partial(\text{robot})/\partial q$, so $\Delta q = -H^{-1}J^{\top}r$ *ascended* the objective. It presents as a negative aperture correlation (-0.74), not as a crash. Fixed; the solver is a comparison baseline and is not in the live engine's path, so no shipped behaviour changed.

Two knobs also have to be set on the *instance* rather than on the module: `ARM_W_ORIENT_FWD` is copied into `self.w_orient_fwd` by `set_profile`, and the posture atlas is disabled by `arm.table = None`. Rebinding the module constant afterwards is silent.

7.5 Repeatability of the harness

The replay is not real time — one tick costs 33 ms of `mj_step` against a 10 ms budget — and the engagement supervisor stamps freshness from the wall clock, so a loaded machine sees a different frame cadence. That is what once made the engaged fraction swing 43–100 % across byte-identical runs (§2.3).

`repeat.py` measures what the ×20 fail-safe timeouts and the engaged-frame mask left behind: the shipped configuration replayed three times over the same six episodes. **Every axis is stable to** CV $\leq 1.9\,\%$, most to under 1 %:

Axis	mean	s.d.	CV
bearing [°]	9.05	0.02	0.2 %
bimanual span [mm]	15.80	0.12	0.8 %
palm normal [°]	2.80	0.05	1.9 %
palm twist [°]	5.00	0.05	0.9 %
aperture [mm]	13.69	0.06	0.5 %
headroom [×]	8.21	0.05	0.6 %
SPARC	−13.48	0.10	0.7 %
own target [mm]	31.33	0.19	0.6 %

Read the tables accordingly: a difference of more than about 2 % is real, and one below it is not. That threshold is on the AGGREGATE, not on a single episode — a single hard clip re-run under machine load moved palm twist 16.1 to 16.9° (5 %), which is why every cell in this document is a median over at least six episodes and never a single replay.

8 Limitations

- **The hand twist is loose, and shipped that way.** See the sweep above: 3.9° to 15.9° at p50, fixable to 2.1° to 4.4° by raising one weight, at a small position cost. Unchanged pending a decision.
- **Transient excursions on large reorientations are real and unfixed.** 3.9 % of frames sit more than 15° off the commanded palm normal, peaking at 42°. G7 localises the cause to palm-roll magnitude rather than to one arm, but does not fix it.

- **Head motion in this corpus is gentle.** Seated tabletop tasks sweep the gaze 0.2° to 16.2° in yaw. Only two clips carry enough yaw amplitude to fit a gain to, so the yaw result rests on n=2; pitch is better supported (n=4). A standing, walking, or looking-around corpus would test the neck far harder.
- **No hardware.** Everything here is the MuJoCo twin. The servo numbers in particular are simulated gains, and the real robot's DaMiao wire caps k_d at ≈ 5.
- **Fingers are converted but not scored.** The 21-landmark skeleton is fed to the keypoint hand IK and the pinch scalar is derived from it, but this study measures the arm, wrist and neck only.
- **Base is stationary.** The clips are seated, so nothing exercises the base-follow path or the local/global frame distinction.
- **One operator per clip, no anthropometry calibration.** The GMR map is root-anchored and scale-normalised, so this is mild, but a per-operator profile is not applied.

References

[1] R. Hoque *et al.*, "EgoDex: Learning Dexterous Manipulation from Large-Scale Egocentric Video," arXiv:2505.11709, 2025.

[2] "Retargeting Matters: General Motion Retargeting for Humanoid Motion Tracking," arXiv:2510.02252, 2025.

[3] N. Hogan and D. Sternad, "Sensitivity of Smoothness Measures to Movement Duration, Amplitude, and Arrests," *Journal of Motor Behavior*, 41(6), 2009.

[4] A. Elias and J. Wen, "Redundancy Parameterization and Inverse Kinematics of 7-DOF Revolute Manipulators," arXiv:2307.13122, 2023.

[5] MABEL controller group, "Front-Workspace Retargeting: the Volumetric Grid Study," `controller/docs/front_workspace_retargeting.tex`.

[6] C. Ionescu, D. Papava, V. Olaru and C. Sminchisescu, "Human3.6M: Large Scale Datasets and Predictive Methods for 3D Human Sensing in Natural Environments," *IEEE TPAMI*, 36(7):1325–1339, 2014.

[7] H. Rhodin *et al.*, "Learning Monocular 3D Human Pose Estimation from Multi-view Images," *CVPR*, 2018. (N-MPJPE: a single fitted scale removes subject height from the error.)

[8] L. Yang *et al.*, "OmniRetarget: Interaction-Preserving Data Generation for Humanoid Whole-Body Loco-Manipulation and Scene Interaction," arXiv:2509.26633, 2025. (Per-frame joint-limit, self-collision and joint-jump violation rates.)

[9] D. Rakita, B. Mutlu and M. Gleicher, "RelaxedIK: Real-time Synthesis of Accurate and Feasible Robot Arm Motion," *RSS*, 2018.

[10] R. Wen *et al.*, "Dexterous Teleoperation of 20-DoF ByteDexter Hand via Human Motion Retargeting," arXiv:2507.03227, 2025. (Inter-fingertip distance tracked over pinch cycles.)

[11] Z.-H. Yin *et al.*, "Geometric Retargeting: A Principled, Ultrafast Neural Hand Retargeting Algorithm," arXiv:2503.07541, 2025.

[12] J. U. Korein, *A Geometric Investigation of Reach*. Cambridge, MA: MIT Press, 1985.

[13] T. Asfour and R. Dillmann, "Human-like Motion of a Humanoid Robot Arm Based on a Closed-Form Solution of the Inverse Kinematics Problem," *IROS*, 2003, pp. 1407–1412.

[14] D. Rakita, B. Mutlu, and M. Gleicher, "A Motion Retargeting Method for Effective Mimicry-based Teleoperation of Robot Arms," *HRI*, 2017, pp. 361–370.

[15] M. Gleicher, "Retargetting Motion to New Characters," *SIGGRAPH*, 1998.